

Домашнее задание 7. JS — основы

JavaScript делает сайт живым и позволяет реагировать на действия пользователя — например: скроллинг, нажатие кнопки, перетаскивание элементов и т.д. JS выполняет код построчно, сверху вниз.

Комментарий — текст в коде, который не исполняется браузером. Обозначается через `//` или `/* ... */`.

Типы данных

- `Number` (число) — может принимать числа, а также значение `NaN` — результат некорректной математической операции, например деления на ноль.
- `String` (строка) — всё, что в кавычках `'...'`, имеет тип `string`.
- `Boolean` (`true/false`) — логический тип.
- `Undefined` — неопределённое значение, например когда переменная объявлена, но значение ещё не присвоено.
- `null` — пустое значение.
- `Symbol` — уникальный идентификатор. Используется редко.
- `BigInt` — большое число. Если нужно записать число больше $2^{53}-1$, используется этот тип — в конце числа ставится `n`: `const number = 124826274746427274647274427n`;
- `Object` — к нему относят: массив, функцию, объект и т.д.

Первые 6 типов используются чаще всего. `NaN` — результат некорректной математической операции; значение специального назначения, но его тип через `typeof` — `number`.

Типизация и typeof

Динамическая (слабая) типизация — тип переменной определяется во время работы программы и соответствует тому, что в ней сейчас находится (число, строка и т.д.).

`typeof` — ключевое слово, которое позволяет определить тип переменной.

```
let value = 123;
console.log(typeof value);
// выведет: number

let value = "123";
console.log(typeof value);
// выведет: string
```

Преобразование типов

Бывает явное — когда программист сам задаёт тип (например через `Number(...)`) — и неявное — когда тип определяет сам JS. Правила приведения типов в JavaScript:

- `+` между значениями приводит всё к типу строка:

```
2 + 3 // 5
"2" + 3 // "23"
```

- Другие математические операции (`-`, `*`, `/`, `<`, `>`) приводят к числу.
- Унарный `+` перед значением пытается преобразовать его в число:

```
+"123" // 123
+true // 1
+false // 0
```

- Двойное отрицание `!!` приводит значение к типу Boolean:
- Ложные (falsy) значения: `0`, `-0`, `0n`, `'`, `null`, `undefined`, `NaN`, `false`.
- Всё остальное, включая `1` и непустую строку — `true`.

Операторы сравнения

Символ `=` — это оператор присваивания, а не сравнения.

- `===` — строгое сравнение: сравнивается и тип данных, и значение.
- `==` — нестрогое сравнение: сравниваются только значения (с приведением типов).

Переменные

Переменная — это имя (ярлык), в котором хранится какое-то значение в коде. JS ищет переменную снизу вверх.

```
let name = "Тимон";
```

- `let` — ключевое слово
- `name` — имя переменной
- `=` — оператор присваивания
- `"Тимон"` — присвоенное значение

То есть мы даём команду: «создай коробку, которая называется `name`, и положи в неё значение Тимон».

Способы объявления переменных:

- `let` — переменная, которую можно изменить в коде после объявления.
- `var` — устаревший способ, имеет глобальную видимость даже внутри блоков.
- `const` — переменная, которая не меняет своего значения после объявления.

Переменная может хранить любой тип данных, а также другие переменные. В JS переменная имеет динамический тип, но не может хранить:

- комментарии,
- конструкции типа блока `if/else`, ключевые слова,
- необъявленные переменные,
- пустой код.

Область видимости переменной — есть три вида:

- Глобальная — доступна везде.
- Функциональная — доступна только внутри функции.
- Блочная — доступна только внутри блока `{ }`.

Этапы создания переменной:

- Объявление — сообщаем JS, что такая переменная существует: `let x;`
- Инициализация — присваиваем переменной значение при создании (в первый раз).
- Присваивание — меняем значение после объявления: `x = 5;`

`let x = 5;` — это объявление + инициализация сразу.

Отличие `var` и `let`: если переменную `var` использовать до объявления, JS не выдаст ошибку, но значение будет `undefined`. С `let` и `const` в этом случае будет ошибка.

Правила именования переменных:

- должно начинаться с буквы (маленькой или большой, но принято — с маленькой);
- может начинаться с нижнего подчёркивания `_`;
- может начинаться с `$`;
- одинаковые имена в разных регистрах — это разные имена;
- нельзя использовать зарезервированные ключевые слова.

Стили именования переменных, принятые в JS:

- `camelCase` — первая буква строчная, начала следующих слов — заглавные.
- `UPPER_CASE` — используется исключительно для физических и математических констант.

Лучшие практики:

- использовать `let` и `const`; `var` — избегать (глобальная видимость);
- давать переменным осмысленные названия;
- следовать стилю `camelCase`.

Конкатенация строк

Конкатенация — склеивание разных строк. Можно двумя способами:

1. Через `+` — старый способ:

```
const firstName = "Тимон";
const myAge = 20;
console.log("Привет, меня зовут " + firstName + ", мне " + myAge + " лет");
// Привет, меня зовут Тимон, мне 20 лет
```

2. Через шаблонную строку с интерполяцией (вставка переменных прямо внутрь строки):

```
const firstName = "Тимон";
const myAge = 20;
console.log(`Привет, меня зовут ${firstName}, мне ${myAge} лет`);
```

```
// Привет, меня зовут Тимон, мне 20 лет
```

Функции

Функция — набор инструкций, собранных под одним именем, которая возвращает нужное значение. Нужна, чтобы не повторять одинаковый код и структурировать программу. Функция характеризуется параметром и аргументом: параметр — это переменная, указанная в круглых скобках при объявлении функции, аргумент — фактическое значение, которое передаётся функции при вызове. У функции может быть несколько параметров. Именованное — как у переменных.

```
function sayHello(name = "Пумба") {  
  console.log(`Привет, ${name}!`);  
  return;  
}
```

```
sayHello("Тимон");  
// Привет, Тимон!
```

- `function` — ключевое слово
- `sayHello` — имя функции
- `name = "Пумба"` — значение параметра по умолчанию
- `(...)` — параметр функции
- `{...}` — тело функции, здесь пишется код
- `sayHello();` — вызов функции (без вызова функция не выполнится)
- `return` — возвращает результат и завершает выполнение функции. Если `return` отсутствует, функция возвращает `undefined`.
- `("Тимон")` — аргумент

Если вызвать функцию без `return` и записать результат в переменную — переменная получит `undefined`. Если при вызове передать больше аргументов, чем параметров в функции — лишние будут проигнорированы JS-ом. Если аргументов меньше, чем параметров — недостающий параметр получит значение `undefined`.

Стрелочные функции

Создаются через переменную. Применяются для сокращения записи. Примеры:

С несколькими параметрами:

```
const sayHi = (name, age) => {  
  console.log(`Привет, ${name}!`);  
}
```

```
sayHi("Влад", 10);
```

С одним параметром:

```
const sayHi = name => {  
  console.log(`Привет, ${name}!`);  
}
```

```
sayHi("Влад");
```

Без параметров:

```
const sayHi = () => {  
  console.log("Привет, мир!");  
}
```

```
sayHi();
```

С возвращением значения:

```
const multiply = (a, b) => {  
  return a * b;  
}
```

```
const result = multiply(10, 9);  
console.log(multiply(5, 6)); // 30  
console.log(result); // 90
```

Укороченная запись для функции с одним выражением:

```
const multiply = (a, b) => a * b;
```

Именованние функций:

- имя должно начинаться с глагола;
- далее можно использовать существительное.

Условная конструкция if / else

Используется для выполнения разных действий в зависимости от условия:

```
if (условие) {  
  // код, если условие true  
} else {  
  // код, если условие false  
}
```

```
const SPEED_OF_LIGHT = 299792458;
```

```
function compareSpeed(speed) {  
  if (speed > SPEED_OF_LIGHT) {  
    console.log("Сверхсветовая скорость");  
  } else if (speed === SPEED_OF_LIGHT) {  
    console.log("Скорость света");  
  } else {  
    console.log("Субсветовая скорость");  
  }  
}
```

`!hasPassport` — логическое отрицание значения переменной.

Тернарный оператор (короткая запись if/else):

условие ? код_если_true : код_если_false

```
let age = 20;  
let access = age >= 18 ? "Разрешён" : "Запрещён";  
console.log(access);
```

`console.log()` используется, чтобы проверить работу кода в консоли браузера.